```
public class BookDataLoader {
  private final BookRepository bookRepository;

  public BookDataLoader(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
  }

  @EventListener(ApplicationReadyEvent.class)
  public void loadBookTestData() {
    bookRepository.deleteAll();                              ❶
    var book1 = Book.of("1234567891", "Northern Lights",
      "Lyra Silverstar", 9.90);
    var book2 = Book.of("1234567892", "Polar Journey",
      "Iorek Polarson", 12.90);
    bookRepository.saveAll(List.of(book1, book2));           ❷
  }
}
```

❶ Deletes all existing books, if any, to start from an empty database

❷ Saves multiple objects at once

DEFINING TRANSACTIONAL CONTEXTS

The repositories provided by Spring Data come configured with transactional contexts for all the operations. For example, all methods in `CrudRepository` are transactional. That means you can safely call the `saveAll()` method, knowing that it will be executed in a transaction.

When you add your own query methods, as you did for `BookRepository`, it's up to you to define which ones should be part of a transaction. You can rely on the declarative transaction management provided by the Spring Framework and use the `@Transactional` annotation (from the `org.springframework.transaction.annotation` package) on classes or methods to ensure they are executed as part of a single *unit of work*.